

HSCTF 2020 Crypto - Smol E

deuterium — 2020-06-06

Canonical HTML: <https://deut-erium.github.io/WriteUps/2020/HSCTF/crypto/Smol'E/2020-06-06-HSCTF-2020-Smol-E.html>

Abstract

CTF writeup: HSCTF 2020's Smol E pads RSA messages on a budget. Coppersmith finds the short-pad difference; Franklin-Reiter finishes the decryption.

Smol E

Challenge

77 Solves



Smol E

398

rsa crypto

Anshul has a message that he wants to send to Disha using RSA. He padded the message with a random padding and sent it to Disha, but she didn't receive the message. However, she was able to receive the message after a second attempt from Anshul. You were able to intercept both of Anshul's ciphertexts, and you are also given Anshul's RSA public key. Can you recover the message?

Author: AC



SmolE.txt

Flag

Submit

N =

16374103928951291344821131644420841508969628115659870754623993906093000530080105004111
05934458085900198112447915951986916531051736670826821921196317026806441235463299073629
13533410257711393278981293987091294252121612050351292239086354120710656815218407878832
422193841935690159084860401941224426397820742950923

E = 3

C1 =

```
11052453979847036661383413388847278106939955208586894208763249935465157511151103606802
18856880924819360603668153227647600050153428761907508779586951683935050277389101011915
28175868547818851667359542590042073677436170569507102025782872063324950368166532649021
589734367946954269468844281238141036170008727208883
```

C2 =

```
42406837735093367941682857892181550522346220427504754988544140886997339709785380303682
47136816810200268289265257729432428691390763561662979048401942164163680549320398914329
85362572966801797451221266550082008296071921912089195257976165232714260921587349720673
87818678258432674493723618035248340048171787246777
```

We have two ciphertexts and N, we can use [Franklin Reiter related message attack](#).

But for that, we need the difference between the two messages. But since the messages are padded with a small pad, here we have another instance of [CopperSmith's Attack](#). We have to use sage to solve this challenge. There are plenty of scripts for this, I used [this nice script](#)

n =

```
16374103928951291344821131644420841508969628115659870754623993906093000530080105004111
05934458085900198112447915951986916531051736670826821921196317026806441235463299073629
13533410257711393278981293987091294252121612050351292239086354120710656815218407878832
422193841935690159084860401941224426397820742950923
```

e = 3

C1 =

```
11052453979847036661383413388847278106939955208586894208763249935465157511151103606802
18856880924819360603668153227647600050153428761907508779586951683935050277389101011915
28175868547818851667359542590042073677436170569507102025782872063324950368166532649021
589734367946954269468844281238141036170008727208883
```

C2 =

```
42406837735093367941682857892181550522346220427504754988544140886997339709785380303682
47136816810200268289265257729432428691390763561662979048401942164163680549320398914329
85362572966801797451221266550082008296071921912089195257976165232714260921587349720673
87818678258432674493723618035248340048171787246777
```

```
PRxy.<x,y> = PolynomialRing(Zmod(n))
```

```
PRx.<xn> = PolynomialRing(Zmod(n))
```

```
PRZZ.<xz,yz> = PolynomialRing(Zmod(n))
```

```
g1 = x**e - C1
```

```
g2 = (x + y)**e - C2
```

```
q1 = g1.change_ring(PRZZ)
```

```
q2 = g2.change_ring(PRZZ)
```

```
h = q2.resultant(q1)
```

```
# need to switch to univariate polynomial ring
```

```
# because .small_roots is implemented only for univariate
```

```
h = h.univariate_polynomial() # x is hopefully eliminated
```

```
h = h.change_ring(PRx).subs(y=xn)
```

```
h = h.monic()
```

```
roots = h.small_roots(X=2**60, beta=0.3)
```

```
assert roots, "Failed1"
```

```

diff = roots[0]
if diff > 2**32:
    diff = -diff
    C1, C2 = C2, C1

r = diff
R.<X> = Zmod(n)[]
f1 = X^3 - C1
f2 = (X + r)^3 - C2

# GCD is not implemented for rings over composite modulus in Sage
def my_gcd(a, b):
    return a.monic() if b == 0 else my_gcd(b, a % b)

mint=-my_gcd(f1, f2).coefficients()[0]
print(mint) # coefficient 0 = -m
print(bytes.fromhex(hex(mint>>1)[2:]))

```

Probably the most annoying part of the challenge was that pad was odd, making me think that the message is gibberish or I may be doing something wrong. Well smashing your head over something is the best part of ctf challenges I guess

Interactive controls are omitted from this PDF. See the canonical HTML version.